

Projectverslag Computer Graphics 2025-2026

Neys Michael (2467626)

14 augustus 2026

1 Inleiding

Dit verslag beschrijft de implementatie van het Computer Graphics project voor het academiejaar 2025-2026. In mijn applicatie heb ik een 3D-omgeving gebouwd met OpenGL (via GLFW en GLAD) waarin een bij (bee) over een traject van samengestelde Bézier-curves vliegt. Dit verslag geeft een gedetailleerd overzicht van de geïmplementeerde functionaliteiten.

2 Geïmplementeerde Functionaliteiten

Om het programma af te sluiten kunnen we gebruik maken van de ‘Q’-toets.

2.1 Curve

Het traject van de bij is opgebouwd uit samengestelde Cubic Bézier-splines aan de hand van controlepunten die voldoen aan de $(3n + 1)$ voorwaarde in de `BezierPath` klasse. Later wordt deze gevisualiseerd met behulp van forward differencing.

2.2 Animatie en Booglengte-parameterisatie

Om te garanderen dat de bij met een constante snelheid over de curve beweegt, is een booglengte-parameterisatie geïmplementeerd:

- **Look-Up Table (LUT):** Tijdens de initialisatie bouwt de functie `buildLUT` een tabel op die de booglengte koppelt aan de t -parameter.
- **Binaire Zoekopdracht ($\mathcal{O}(\log N)$):** Om de gewenste afstand om te zetten naar de bijbehorende t -parameter gebruikt `getTForDistance` het C++ `std::lower_bound` algoritme. Dit reduceert de zoektijd in de LUT van een lineaire $\mathcal{O}(N)$ naar een logaritmische $\mathcal{O}(\log N)$ complexiteit.
- **Lineaire Interpolatie:** De exacte t -waarde tussen twee LUT-ijkpunten wordt via lineaire interpolatie berekend. Dit voorkomt schokkerige bewegingen bij lage LUT-resoluties.

2.3 Modellen, textures & terrein

De 3D-wereld is opgebouwd uit modulaire sub-systemen:

- **Voxel Terrein:** Een procedureel gegenereerd hoogte-terrein via de `Terrain` klasse.
- **Skybox:** Een `Skybox` klasse die 6 cubemap-texturen inlaadt via `stb_image`. Tijdens het renderen wordt de translatiematrix uit de View-matrix gestript en wordt de dieptetest ingesteld op `GL_LEQUAL` en `glDepthMask(GL_FALSE)`, zodat de achtergrond altijd achter alle 3D-objecten blijft liggen.

- **3D-modellen:** Er wordt een statische ‘Minecraft village / station’ en een ‘Minecraft Bee’ (bij) ingeladen uit 2 aparte glb bestanden die ook meteen de textures bevatten. De bij volgt de Bézier curve aan een constante snelheid.

2.4 Visualisatie

Om te wisselen naar het secundaire traject kunnen we gebruik maken van de ‘**T**’-toets of onze picking interactie uitvoeren dat ook de track verandert naar het andere spoor.

- **TrackRenderer & Pollen:** De `TrackRenderer` genereert via Forward Differencing een visuele groene hulplijn op de GPU. Deze klasse berekent ook de willekeurig verdeelde transformatiematrices voor stuifmeelkorrels langs de route.

2.5 Camera’s

Er zijn twee camera-modi geïmplementeerd die gewisseld kunnen worden met de ‘**C**’-toets:

- **Free-look/fly camera:** Hiermee kan de hele scène vrij verkend worden.
 - W, A, S, D: Navigatie (vooruit, links, achteruit, rechts).
 - Spatiebalk: Verticaal omhoog bewegen.
 - Control (Ctrl): Verticaal omlaag bewegen.
 - Shift: Versnellen/sprinten in de richting waarin je momenteel beweegt.
- **First-person camera (Bij):** Deze camera is vastgemaakt aan de bij. De View-matrix wordt berekend op basis van de actuele positie op de curve en de richting naar het volgende punt op het traject (`glm::lookAt`).

Beide camera-modi ondersteunen in- en uitzoomen met respectievelijk ‘scroll omhoog’ en ‘scroll omlaag’.

2.6 Belichting

Per-pixel belichting met point lights waarvan de intensiteit afneemt over afstand:

- **Per-pixel Belichting:** Geïmplementeerd in de fragment shader (`lighting.frag`); ambient, diffuse en specular. De berekening gebeurt per pixel op basis van de genormaliseerde normaal- en kijkvectoren.
- **Lokale Lichtbronnen (18 point lights):** Er zijn 18 lichtbronnen langs het parcours en rond het station geplaatst.
- **Afstandsverzwakking (Attenuation):** De invloed van elke lantaarn neemt realistisch af met de afstand via de formule:

$$\text{Attenuation} = \frac{1.0}{\text{constant} + \text{linear} \cdot d + \text{quadratic} \cdot d^2}$$

- **Lichtbeheer & Optimalisatie:** De `LightManager` klasse beheert de uniform-parameters. Om rendering-overhead te vermijden, worden de uniforme namen vooraf gecachet. Met de ‘**R**’-toets kunnen de lampen getoggled worden.

2.7 Convolutie en Post-processing (Bloom)

Post-processing wordt toegepast door de scène te renderen naar een Framebuffer Object (FBO):

- **Convolutie:** Via de toetsen **1**, **2** en **3** kan geschakeld worden tussen de standaardweergave, een Gaussian Blur en Edge Detection (Laplaciaan/Sobel).
- **Bloom:** Lantaarns krijgen een realistisch glow-effect via een brightness-extractie shader (`bloom_bright.frag`). Vervolgens wordt deze buffer via 'ping-pong' framebuffers in meerdere passes ge-blurred en samengevoegd met de originele scène. Dit kan getoggled worden via de **'B'-toets**.

2.8 Chroma-keying

Via de `ChromaKey` klasse is een textured quad aan de scène toegevoegd:

- De overlay-textuur wordt ingeladen met `stb_image` en gerenderd met de `chromaKey.frag` shader.
- Chroma-keying detecteert de specifieke groene achtergrondkleur en verwijdert deze met het GLSL `discard` commando.
- Om beeldvervalsing te voorkomen worden de afmetingen van het vlak dynamisch geschaald op basis van de aspect ratio van de afbeelding.
- Standaard is staat de chroma-keying aan maar is de overlay in zijn geheel niet aangezet, om de overlay aan of uit te schakelen kunnen we gebruik maken van de **'O'-toets**. Voor chroma-keying aan of uit te schakelen kunnen we gebruik maken van de **'G'-toets**.

2.9 GPU Color-based Picking (Interactie)

Voor de interactie met objecten is gekozen voor color-based picking (Off-screen Rendering) in plaats van traditionele raycasting:

- **Picking FBO & Shader:** Er is een dedicated Framebuffer Object (`pickingFBO`) en een specifieke `pickingShader`.
- **Off-screen Rendering:** Bij een muisklik worden de interactieve meshes (zoals de lamp-mesh) off-screen naar de FBO getekend met een unieke identificatiekleur (puur rood: `RGB(255, 0, 0)`). De belichting en textures worden hierbij overgeslagen.
- **Pixel Uitlezen (`glReadPixels`):** De applicatie leest direct de kleurwaarde uit van de enkele pixel in het midden van de buffer (`screenWidth / 2`, `screenHeight / 2`).
- **Hit Detection:** Als de uitgelezen kleur exact overeenkomt met de rode id-kleur (`pixel[0] == 255`), wordt een succesvolle interactie gedetecteerd. Dit roept de `toggleLamp()` en `toggleTrack()` functies aan.

3 Niet-geïmplementeerde Functionaliteiten

Alle basisfunctionaliteiten uit de opgave werden geïmplementeerd.

4 Planning

Ik heb verdergewerkt aan het project via Git. In de onderstaande tabel wordt de tijdsbesteding (per dag/periode en het aantal uren) inzichtelijk gemaakt aan de hand van onze bijgehouden urenregistratie, zoals vereist in de opgave. Dit is enkel een indicatie naar het verbeteren van het

project en dus niet het volledig maken ervan aangezien we verder mochten werken met de reeds bestaande code dat we in groep mochten maken.

Datum	Tijd	Taak / Beschrijving
16 juli	30 min	Repository schoonmaken en cotrols beter documenteren
17 juli	10 min	Bestandsstructuur en imports updaten
17 juli	4 uur	Track updaten zodat we Bézier splines maken ipv 1 ding berekenen
17 juli	30 min	Secundaire track toevoegen
17 juli	30 min	Forward differencing toevoegen
22 juli	1 uur	Gebruik maken van standard descturctors
22 juli	1 uur	Chroma-keying een overlay met toggles maken
22 juli - 2 augustus	3 uur	Documentatie
5, 13 augustus	2 uur	Verslag
13 augustus	1 uur	Video (opname + editen)

Tabel 1: Overzicht van de daadwerkelijk gevolgde planning en bestede tijd.